Lab Num: Four **Topic: Bare-Metal Programming : Loading and Running a Custom Binary from U-Boot**

- Write a complete 64-bit AArch64 bare-metal application from scratch that blinks an LED connected to a GPIO pin on the Raspberry Pi 3B+. You must compile and link it correctly, copy it to the boot partition, load it into RAM using your custom U-Boot (from Lab 02/03), and execute it.
- **What You Must Deliver**
    - Source code
    - startup.s (ARM assembly entry point)
    - main.c (C code that controls the GPIO)
    - linker.ld (custom linker script with correct load address)
    - Makefile (fully automated build → final .img binary)
    - Final binary file named <your_binary>.img (or yourname_blinky.img)
    - Hardware proof:
        - Clear photo of your LED + resistor wiring (GPIO 26 or GPIO 23)
        - Short video (5–15 seconds) OR at least 3 timed photos showing: → U-Boot serial console executing LED visibly ON → LED visibly OFF (blinking)
- **Requirements & Rules**
    - The LED must blink at a visible frequency.
    - The program must never crash or return — include proper hang loop if main exits
    - Brief explanation of the load address you chose and why
    - Why do we need a startup.s file at all? Can't we just write everything in C?